SCHEDULEPOINT

From iSchedule.md Research to a Working App

An integrated Claude Code + Claude for Chrome playbook

Use one Claude Code session to explore https://ischedule.md in read-only mode, save structured research automatically, design an original product called SchedulePoint, and build it one tested vertical slice at a time.

The operating model

Claude Code is the single workspace. Its Chrome extension autonomously navigates iSchedule.md to observe behaviour. Claude Code writes every report, updates indexes and the checklist, validates the files, and stops for the next phase. You do not copy research between tools or run a separate capture prompt.

Read-only boundary

Allowed: navigation links, tabs, menus, pagination, view selectors, non-saving search and filters, accordions, drawers, modals, and opening forms to inspect them. Forbidden: save or submit; publish; approve; accept; reject; withdraw; claim; transfer; invite; message; notify; upload; import; delete; change settings; impersonate; or confirm any action that might change server-side data. If safety is uncertain, do not click.

Model guide

Use the best available Sonnet-class model for routine exploration, documentation, and implementation. Use the strongest Opus-class model with high effort for architecture, state machines, concurrency, security, and difficult debugging. Use Codex as the independent reviewer.

Automatic completion rule

Every phase prompt must create or update its named file, merge by stable ID, update manifest.json, source-page-index.md, evidence-register.md, unresolved-questions.md, and MASTER-CHECKLIST.md, run validation, checkpoint Git when available, and then stop. A phase is complete only when those checks pass.

Master Checklist

This is a native Google Docs checklist. Claude Code also maintains the matching MASTER-CHECKLIST.md inside the workspace.

- [x] ~~Phase 1 — Create the integrated research workspace~~
- [x] ~~Phase 2 — Map the complete iSchedule.md application~~
- [x] ~~Phase 3 — Map roles, accounts, permissions, and organization structure~~
- [x] ~~Phase 4 — Document user-facing workflows~~
- [x] ~~Phase 5 — Investigate the master schedule and manual scheduling~~
- [x] ~~Phase 6 — Investigate scheduling configuration, rules, and builds~~
- [x] ~~Phase 7 — Map requests, vacation, opportunities, swaps, and approvals~~
- [ ] ~~Phase 8 — Investigate the picklist system~~ ✅
- [ ] ~~Phase 9 — Cover reports, documents, contacts, settings, and all other modules~~ ✅
- [ ] ~~Phase 10 — Check responsive and accessibility behaviour~~ ✅
- [ ] ~~Phase 11 — Capture authorized technical and network observations~~ ✅
- [ ] ~~Phase 12 — Build the edge-case and QA catalogue~~ ✅
- [ ] ~~Final coverage audit — Reconcile research against iSchedule.md~~ ✅
- [ ] Phase 13 — Consolidate the research
- [ ] Phase 14 — Reconcile the product and answer blocking decisions
- [ ] Phase 15 — Create the private SchedulePoint repository
- [ ] Phase 16 — Design and document the architecture
- [ ] Phase 17 — Run the independent Codex architecture review
- [ ] Phase 18 — Resolve findings and create the vertical-slice roadmap
- [ ] Phase 19 — Design the original SchedulePoint interface
- [ ] Phase 20 — Scaffold and verify the project foundation
- [ ] Phase 21 — Implement one vertical slice at a time
- [ ] Phase 22 — Review implementation
- [ ] Phase 23 — Compare behaviour and run hostile QA
- [ ] Phase 24 — Stabilize SchedulePoint and prepare private beta

Phase 1 — Create the integrated research workspace ✅

Claude Code • Sonnet-class • Output: schedulepoint-research/

What happens

Run this once. Claude Code creates every folder, index, rule file, validation command, and native project checklist needed by later phases.

COPY-PASTE PROMPT — Sonnet 5

```
Create the SchedulePoint research workspace now; perform the filesystem work rather than describing it.

Create:
schedulepoint-research/
  screenshots/navigation/
  screenshots/schedule/
  screenshots/requests/
  screenshots/vacation/
  screenshots/picklist/
  screenshots/admin/
  screenshots/errors/
  scripts/

Create starter files:
RESEARCH-RULES.md
MASTER-CHECKLIST.md
reports/manifest.json
reports/source-page-index.md
reports/evidence-register.md
reports/unresolved-questions.md

RESEARCH-RULES.md must state that Claude for Chrome may autonomously navigate https://ischedule.md in read-only mode using links, tabs, menus, pagination, non-saving search and filters, view selectors, drawers, and modals. It must prohibit every server-side mutation, message, upload, import, approval, acceptance, publication, deletion, setting change, and impersonation. If safety is uncertain, do not click.

Create a validation command that checks required files, non-empty reports, stable IDs, confidence labels, and absence of obvious secrets or sensitive data. Initialize Git if appropriate. Report the tree created, run validation, and stop.
```

Phase 2 — Map the complete iSchedule.md application ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/01-application-map.md

What happens

Claude autonomously traverses the source site, without being limited to predefined modules, and builds the retrieval index used by every later phase.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md and obey it. Open https://ischedule.md with the Chrome extension and autonomously explore as broadly and systematically as possible in read-only mode.

Discover every accessible page, route, navigation item, tab, menu, modal, drawer, popover, help surface, role clue, workflow entry point, report, setting, integration, empty/loading/error state, and responsive variation. Do not restrict discovery to categories already known.

For every screen create a stable screen ID and record name, URL, navigation path, purpose, likely roles, visible sections, controls, data shown, related screens, observable states, unanswered questions, and evidence references. Create a coverage map showing visited, partially inspected, inaccessible, and unresolved areas. Identify promising follow-up paths for later phases.

Never perform a mutating action. Stop before any save, submit, confirmation, message, upload, import, approval, acceptance, publication, deletion, setting change, or uncertain action.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 3 — Map roles, accounts, permissions, and organization structure ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/02-roles-and-permissions.md

What happens

Claude revisits relevant screens and builds a confidence-rated permission matrix.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md and reports/01-application-map.md. Autonomously inspect role, account, organization, group, site, department, user administration, proxy, locum, scheduler, special-permission, and group-switching surfaces in read-only mode.

Document every observed and inferred role; membership structure; group switching; navigation differences; view/create/edit/archive/publish/approve/picklist/message/impersonation powers; special flags; activation states; and uncertain permissions. Build a role-permission matrix with evidence IDs and confidence. Normal authorized navigation is allowed; never bypass access controls, impersonate, invite, activate, deactivate, save, or submit.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 4 — Document user-facing workflows ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/03-user-workflows.md

What happens

Claude records exact observable workflow sequences while stopping before consequences.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md and the existing indexes. Autonomously inspect all ordinary user and physician workflows in read-only mode, including login/reset surfaces, group switching, personal and organization schedules, date ranges, shift details, ON/OFF requests, request status, opportunities, give-away, swaps, on-call and daily assignments, contacts, documents, profile, notification preferences, proxies, calendar subscription, and every other user workflow discovered.

For each workflow record stable ID, actor, preconditions, start screen, exact navigation, fields/defaults, validation, system responses, states, notifications that would be triggered, alternate/failure paths, edge cases, evidence, and unanswered questions. Open forms and confirmation dialogs only to inspect them; never complete consequential actions.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 5 — Investigate the master schedule and manual scheduling ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/04-master-schedule.md

What happens

Claude maps the central scheduling interface, its views, editing model, and publication rules without changing schedules.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md. Autonomously inspect the master schedule in read-only mode. Cover Date, Staff, Shift, and any newly discovered views; ranges, filters, paging, highlighting, legends, summaries, cell/header interactions, assignments, credits, OFF/vacation, counts, balances, notes, holidays, events, requests, daily picks, picklist controls, and batch tools.

Document draft/published behaviour, unsaved changes, save with/without notifications, discard, post-publication edits, staging, publication prerequisites, incomplete schedules, locks, user visibility, and rollback/unpublish evidence. Open menus, drawers, editors, and confirmation dialogs to inspect them, but never save, publish, notify, delete, move, batch-edit, or change schedule data. Add Given/When/Then acceptance criteria.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 6 — Investigate scheduling configuration, rules, and builds ✅

Claude Code + Chrome extension • Sonnet; Opus for synthesis • Output: reports/05-scheduling-engine.md

What happens

Claude maps administrative configuration and the inferred scheduling engine without running it.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md. Autonomously inspect every scheduling-administration surface: group settings, builds, build management, planner, Staff Shift FTE, statistics, staff/groups, shift definitions/groups, valid groups, pattern rules, staff rules, optimization settings, generation actions, build history, statuses, errors, and anything else discovered.

Record fields, types, defaults, required status, validation, options, relationships, help text, dependencies, save/delete behaviour, future-versus-existing schedule effects, and terminology. Determine observed inputs, periods, demand, staff targets, hard/soft rules, weights, equations, patterns, requests/vacation/manual-assignment effects, initiation, outputs, failures, regeneration, review, and publication. Never save configuration or run/regenerate a build.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 7 — Map requests, vacation, opportunities, swaps, and approvals ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/06-requests-vacation-opportunities.md

What happens

Claude produces state machines for the main request and exchange lifecycles.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md. Autonomously inspect ON/OFF and shift-group requests; vacation selection, quota, grant, approval, transfer, and administration; opportunity creation/acceptance; locum restrictions; confirmations; swaps; and transfer approvals.

For each lifecycle document rules, dates/deadlines, fields, states, transitions, actors, approval, notifications, cancellation, rejection, expiry, edit/delete, assignment/statistics effects, conflicts, and races. Include Mermaid state diagrams and acceptance criteria. Open read-only details and dialogs, but never submit, approve, accept, reject, withdraw, claim, transfer, delete, or notify.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 8 — Investigate the picklist system ✅

Claude Code + Chrome extension • Sonnet; Opus for state model • Output: reports/07-picklist-system.md

What happens

Claude maps preparation, real-time execution, escalation, and completion without starting a live picklist.

COPY-PASTE PROMPT — Sonnet 5

```
Read RESEARCH-RULES.md. Autonomously inspect the picklist from physician and scheduler perspectives.

Cover preparation: dates, room/slate import shape, manual rooms, ordering, staff import, pick order, lock/unlock, comments. Execution: start/pause/resume/complete controls, current picker, timers, room selection/confirmation, failures/retries, skips/exclusions, proxy, automatic advancement, intervention. Notifications: email/SMS/calls, mandatory/personal ladders, timing, defaults/overrides, failures. Real time: monitoring, live updates, refresh, connection loss, multiple browsers, concurrent selection, race protection, alerts. Completion: assignments, distribution, audit, statistics, correction.

Never start, pause, resume, complete, import, select a room, confirm, notify, or change a picklist. Record only the presence/location—not the contents—of patient or health information.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.
```

Phase 9 — Cover supporting modules and anything not yet classified ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/08-supporting-modules.md

What happens

Claude closes breadth gaps and captures every module not already owned by another report.

COPY-PASTE PROMPT — Sonnet 5

```
Read RESEARCH-RULES.md and the coverage map. Autonomously inspect reports/statistics, printing, exports, contacts, email/SMS composition, documents/files, profile, notification settings, group defaults, help/tours/support, integrations, and every discovered module not yet adequately documented.

For reports record inputs, filters, calculations, grouping, sorting, print/export/share, permissions, and empty states. For documents record categories, file constraints, naming, access, upload/download/delete flows, and audit behaviour. For all other modules use stable IDs and the standard screen/workflow/rule format. Never send, share, upload, download sensitive files, delete, or save changes.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.
```

Phase 10 — Check responsive and accessibility behaviour ✅

Claude Code + Chrome extension • Sonnet-class • Output: reports/09-responsive-accessibility.md

What happens

Claude tests presentation changes and observable accessibility behaviour without mutating data.

COPY-PASTE PROMPT — Sonnet 5

Read RESEARCH-RULES.md. Revisit representative iSchedule.md screens at desktop, tablet, and phone widths. Document navigation changes, calendar adaptation, tables, modals, scrolling, touch targets, density, truncation, and print behaviour.

Perform read-only keyboard checks for focus order, visible focus, menus, dialogs, Escape, labels, validation exposure, and skip links. Record observable semantic hints, contrast concerns, and screen-reader questions. Separate observations from recommendations and do not claim WCAG compliance. Do not submit forms or activate consequential controls.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.

Phase 11 — Capture authorized technical and network observations ✅

Claude Code + Chrome extension / DevTools • Sonnet-class • Output: reports/10-technical-observations.md

What happens

Claude records technical evidence only where browser-visible and authorized.

COPY-PASTE PROMPT — Sonnet 5

```
Read RESEARCH-RULES.md. Using authorized browser-visible information and sanitized network evidence, document route/API resource patterns, HTTP methods, sanitized payload shapes, pagination/filter/sort, validation errors, high-level session behaviour, uploads/downloads/calendar feeds, real-time technology, polling/WebSockets, caching, browser storage, URL state, performance concerns, broken endpoints, and client/server responsibilities.

Never expose tokens, cookies, credentials, patient data, personal data, private keys, or secret URLs. Never replay, modify, or manually invoke requests. Label every technical claim OBSERVED, INFERRED, or UNRESOLVED. Explain behaviour SchedulePoint must independently support; do not copy proprietary implementation.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.
```

Phase 12 — Build the edge-case and QA catalogue ✅

Claude Code + Chrome extension • Sonnet; Opus for prioritization • Output: reports/11-edge-cases-and-qa.md

What happens

Claude records naturally observable edge cases and specifies the rest for later SchedulePoint testing.

COPY-PASTE PROMPT — Sonnet 5

```
Read RESEARCH-RULES.md and all reports. Record naturally occurring read-only edge cases visible in iSchedule.md. Do not create test data or provoke failures on the source site.

Specify future SchedulePoint tests for empty/unassigned/overlapping/duplicate schedules; inactive users and locums; multi-group membership; date boundaries, deadlines, DST, overnight shifts, leap years; vacation oversubscription; incomplete publication; post-publication edits; concurrent opportunity acceptance, room picks, and schedule edits; refresh/back/direct URLs/session expiry/network loss; invalid uploads and duplicate imports; large organizations; long names; missing contact data; notification failures; mobile; keyboard; screen readers; and contrast.

For every case record ID, preconditions, steps, expected result, observed source behaviour if naturally available, risk, match-versus-improve recommendation, and priority.

When finished, write the complete report to the specified path. Use stable IDs and label each claim OBSERVED, INFERRED, or UNRESOLVED. Merge by stable ID instead of duplicating earlier findings. Update reports/manifest.json, reports/source-page-index.md, reports/evidence-register.md, reports/unresolved-questions.md, and MASTER-CHECKLIST.md. Validate that the report is complete, non-empty, and free of credentials, tokens, cookies, patient information, and unnecessary personal data. Create a Git checkpoint if available. Mark this phase complete only if validation passes, then stop and tell me only what was saved and what needs my attention.
```

Phase 13 — Consolidate the research

Claude Code • Opus-class • Output: reports/12–16

What happens

Run the five consolidation tasks in order. Claude writes each file directly and updates the project index after every successful output.

COPY-PASTE PROMPT — Opus 4.7

Run these five consolidation tasks sequentially, using only reports/01 through 11:

Product glossary — Create reports/12-product-glossary.md with term, definition, related IDs, usage, and confidence. Preserve genuine contradictions.

Feature inventory — Create reports/13-feature-inventory.md with feature ID, users, purpose, dependencies, workflow, rules, statuses, permissions, notifications, edge cases, confidence, and MVP priority.

Domain model — Create reports/14-domain-model.md with entities, fields/types, relationships, tenant ownership, status, audit, retention, sensitivity, confidence, and Mermaid ERD.

State machines — Create reports/15-state-machines.md for schedule build/publication, requests, vacation, opportunities, swaps, picklists, room selection, notification escalation, users, and documents.

Research completion — Create reports/16-research-completion.md with confirmed capabilities, unresolved questions, unsafe-to-test areas, apparent defects, redesign needs, blockers, MVP scope, and post-MVP scope.

After each file, update all indexes and validation. At the end update MASTER-CHECKLIST.md, checkpoint Git, stop, and report only failures or decisions.

Phase 14 — Reconcile the product and answer blocking decisions

Claude Code • Opus-class • Output: docs/product/

What happens

Claude normalizes the research into an approved product definition without inventing source behaviour.

COPY-PASTE PROMPT — Opus 4.7

```
Read reports/01 through 16. Create docs/product/product-definition.md, permissions-matrix.md, business-rules.md, product-decisions.md, and mvp-scope.md.

Reconcile contradictions without inventing facts. Separate observed source behaviour from SchedulePoint recommendations. For every product-owner decision show the question, options, consequences, recommendation, and whether it blocks architecture. Update MASTER-CHECKLIST.md and stop for my answers before architecture.
```

Phase 15 — Create the private SchedulePoint repository

Claude Code • Sonnet-class • Output: SchedulePoint repository

What happens

Claude moves approved research into a version-controlled project without starting application development.

COPY-PASTE PROMPT — Sonnet 5

```
Create or initialize a private-ready SchedulePoint repository with README.md, CLAUDE.md, AGENTS.md, docs/research, docs/product, docs/architecture, docs/implementation, apps, and packages. Copy—not rewrite—the validated research into docs/research and the reconciled product files into docs/product. Preserve provenance and stable IDs. Do not scaffold application code. Run validation, commit the documentation baseline, update MASTER-CHECKLIST.md, and stop.
```

Phase 16 — Design and document the architecture

Claude Code • Opus-class, high effort • Output: docs/architecture/

What happens

Claude produces the domain, tenancy, authorization, state, concurrency, audit, and deployment design—without coding.

COPY-PASTE PROMPT — Opus 4.7

```
Read docs/research and docs/product. Design the clean-room SchedulePoint architecture. Create documents for overview, domain model, PostgreSQL/ERD, tenancy, authorization, state machines, scheduling engine, picklists, notifications, imports, testing, security, deployment, and decisions-needed. Draft CLAUDE.md and AGENTS.md.

Keep patient information out of MVP unless explicitly required. Enforce tenant boundaries server-side and in the database where practical. Preserve published history, protect transitions server-side, design race protection and idempotency, use maintained libraries, avoid unnecessary microservices, and label assumptions. Do not implement code. Validate cross-document consistency, update the checklist, checkpoint, and stop.
```

Phase 17 — Run the independent Codex architecture review

Codex • Frontier coding model, high reasoning • Output: docs/architecture/codex-review.md

What happens

Codex challenges the design independently and makes no changes.

COPY-PASTE PROMPT — Codex GPT-5.6

```
Act as an independent principal architect. Read docs/research, docs/product, docs/architecture, CLAUDE.md, and AGENTS.md. Do not modify files.

Review missing entities/relationships; tenant/group/site boundaries; authorization; transition bypasses; published history; concurrent edits; opportunity and room-pick races; notification retries; import idempotency; time zones and overnight shifts; audit; sensitive information; reporting consistency; scheduling-engine design; unsupported claims; and excess complexity.

Rank findings Critical/High/Medium/Low. For each give file, problem, realistic failure, recommended change, and detecting test. Finish Approved / Approved after changes / Redesign required. Save as docs/architecture/codex-review.md and stop.
```

Phase 18 — Resolve findings and create the roadmap

Claude Code • Opus then Sonnet • Output: docs/implementation/

What happens

Claude resolves architecture findings and converts the approved MVP into small vertical slices.

COPY-PASTE PROMPT — Opus 4.7

```
Read docs/architecture/codex-review.md. Resolve valid Critical and High findings, record agree/disagree reasoning, update affected architecture and test requirements, and create review-resolution.md.

Then create docs/implementation/roadmap.md, milestones.md, backlog.md, and mvp-acceptance-tests.md. Use small vertical slices covering organizations/groups, accounts, locations, shift types, periods, manual shifts/assignments, draft/publication, schedule view, basic ON/OFF requests, opportunities, acceptance, optional confirmation, audit, and email. Explicitly defer optimization, patient-level OR data, voice/SMS, advanced vacation bidding, complex reports, documents, and full picklist automation unless approved. Update the checklist, checkpoint, and stop.
```

Phase 19 — Design the original SchedulePoint interface

Claude Code + Figma if desired • Sonnet or Opus • Output: docs/design/ux-brief.md

What happens

Claude creates an original UX brief and wireframe plan rather than copying iSchedule.md.

COPY-PASTE PROMPT — Sonnet 5

```
Using the approved product and MVP, create docs/design/ux-brief.md for an original SchedulePoint interface. Define navigation; desktop/mobile layouts; schedule; editor; shift details; requests; opportunities/transfers; user administration; empty/loading/error/confirmation states; accessibility; components; status language; colors; and responsive behaviour.

For each screen give purpose, hierarchy, components, actions, errors, permissions, and mobile adaptation. Do not copy iSchedule.md branding, text, icons, or visual composition. Update the checklist and stop.
```

Phase 20 — Scaffold and verify the project foundation

Claude Code • Sonnet-class • Output: Runnable project foundation

What happens

Claude creates only the technical foundation and proves it works.

COPY-PASTE PROMPT — Sonnet 5

Read all approved product, architecture, implementation, and repository instructions. Implement only the foundation milestone: repository structure, TypeScript, framework, PostgreSQL/ORM/migrations, local environment, authentication foundation, tenancy and authorization utilities, audit foundation, unit/integration/Playwright, lint/format/typecheck, CI, seed data, setup docs, and example environment file.

Do not implement scheduling features. Do not hard-code secrets or weaken tenant isolation. Run and fix every applicable check, inspect startup, document deviations, update the checklist, checkpoint, and stop.

Phase 21 — Implement one vertical slice at a time

Claude Code • Sonnet; Opus for concurrency/domain changes • Output: One tested backlog item

What happens

Run the same prompt once per approved backlog item. Never bundle unrelated slices.

COPY-PASTE PROMPT — Sonnet 5

```
Implement backlog item [ID — NAME].

Read the exact item and linked requirements first. Include all applicable migration, domain logic, server-side tenant authorization, API/server action, validation, UI, loading/empty/error states, audit event, unit test, integration test, Playwright test, and documentation update.

Do not implement unrelated items, silently change business rules, rely on UI-only authorization, or duplicate domain logic. Run lint, typecheck, tests, the new Playwright flow, and production build as applicable. Inspect desktop/mobile behaviour, fix failures, update the checklist and backlog status, checkpoint, and stop.
```

Phase 22 — Review implementation independently

Codex, then Claude Code • High reasoning • Output: docs/reviews/

What happens

Codex finds issues first; Claude Code fixes only after the review is saved.

COPY-PASTE PROMPT — Codex GPT-5.6

```
Review implementation changes since the previous approved milestone. Do not modify files.

Check product compliance, tenant isolation, server authorization, transitions, races, duplicate actions, audit gaps, date/time handling, database consistency, errors, accessibility, tests, edge cases, and complexity. Identify UI restrictions not enforced by the backend.

Rank findings and include file/code area, failure scenario, recommended fix, and regression test. Save as docs/reviews/[MILESTONE]-codex-review.md. Then have Claude Code resolve valid Critical/High findings with regression tests, update the checklist, checkpoint, and stop.
```

Phase 23 — Compare behaviour and run hostile QA

Claude Code + Chrome extension + Playwright • Opus-class • Output: docs/testing/mvp-qa-report.md

What happens

Claude re-observes source behaviour read-only and attacks only the replacement application.

COPY-PASTE PROMPT — Opus 4.7

```
Use Claude for Chrome to re-navigate relevant iSchedule.md workflows in strict read-only mode. Use Playwright to exercise only SchedulePoint. Document matching behaviour, missing behaviour, intentional differences, and accidental differences.

Then test SchedulePoint for unauthorized navigation, cross-tenant IDs, duplicates, simultaneous acceptance/edits, incomplete publication, post-publication edits, inactive assigned users, overlaps, invalid dates, DST, refresh, failed jobs, repeated notifications, expiry, archived dependencies, empty/large organizations, slow networks, mobile, and keyboard use.

For each failure record reproduction, root cause, severity, and a failing regression test. Fix Critical/High defects without weakening tests. Produce docs/testing/mvp-qa-report.md, update the checklist, checkpoint, and stop.
```

Phase 24 — Stabilize SchedulePoint and prepare private beta

Claude Code + Playwright + Sentry • Sonnet; Opus/Codex final review • Output: Release checklist

What happens

Claude adds monitoring and operational safeguards and stops before deployment.

COPY-PASTE PROMPT — Sonnet 5

```
Prepare SchedulePoint for private beta. Add production-safe error monitoring with sensitive-data scrubbing; critical-path Playwright tests; accessibility checks; visual regression; backup/restore documentation; migration rollback procedures; rate limits; job monitoring; notification tracing; and deployment runbooks.

Never record names, schedules, messages, patient information, tokens, or form contents in logs, analytics, or replay. Produce a private-beta release checklist and unresolved-risk register. Run final security and recovery reviews, update the master checklist, checkpoint, and stop. Do not deploy until I approve the checklist.
```

Appendix — Definition of done for every implementation slice

- Matches approved acceptance criteria.
- Enforces tenant and role rules on the server.
- Validates state transitions, idempotency, and concurrency.
- Creates required audit events.
- Includes loading, empty, success, error, and permission-denied states.
- Includes unit, integration, and Playwright coverage proportional to risk.
- Passes lint, type checking, tests, and production build.
- Has been inspected at desktop and mobile sizes.
- Updates documentation, backlog status, and the master checklist.
- Contains no secrets, live customer data, or patient information.

Closing rule

One Claude Code session. One phase prompt at a time. Automatic saving and indexing. Read-only exploration of iSchedule.md. Original SchedulePoint design and code. Independent review before each major commitment.